# Buscador de duplicados

Guía del código — cómo funciona por dentro

Recorrido por el código para quien quiera entenderlo o aprender a partir de él: la arquitectura, los algoritmos (las 3 pasadas del escaneo, dHash, BK-tree, Union-Find), el patrón de hilos de la interfaz, el empaquetado y cómo se prueba todo.

Aplicación de escritorio para Windows · Python + CustomTkinter
Documento generado el 21/10/2026

# Índice

**1. Cómo leer este proyecto . . . 3**

1.1. Mapa de módulos . . . 3

**2. El núcleo, explicado . . . 5**

2.1. Detectar duplicados exactos: tres pasadas . . . 5

2.2. El hash: BLAKE2b y lectura en bloques . . . 5

2.3. El filtro: lista blanca y lista negra . . . 5

2.4. La decisión: qué copia se conserva . . . 6

2.5. Fotos parecidas: dHash + BK-tree + Union-Find . . . 6

2.6. Mover archivos con seguridad . . . 7

2.7. Historial y deshacer . . . 7

**3. La interfaz, explicada . . . 8**

3.1. CustomTkinter y el arranque . . . 8

3.2. El patrón hilo + cola + after() (el más importante) . . . 8

3.3. Las tres pestañas . . . 9

3.4. Tema claro / oscuro . . . 9

3.5. Modo Sencilla / Avanzada . . . 9

3.6. La tabla (ttk.Treeview) . . . 9

3.7. Miniaturas e imágenes en Tkinter . . . 9

**4. Empaquetado . . . 10**

**5. Cómo se prueba . . . 10**

**6. Añadir una función nueva: receta . . . 10**

# 1. Cómo leer este proyecto

En esta guía te explico cómo está hecho el código, para que puedas entenderlo y aprender de él, igual que yo he ido aprendiendo. Está en la raíz del repositorio y lo dividí en tres partes bien separadas:

| Carpeta | Qué contiene | Regla |
|---|---|---|
| core/ | Toda la lógica: escanear, comparar, mover, deshacer. | No importa NADA de la interfaz. Se prueba entero por consola. |
| ui/ | La ventana y todos sus widgets (CustomTkinter). | Solo pinta y reacciona a clics; delega la lógica en core/. |
| pruebas/ | 15 suites + pruebas unitarias; se ejecutan con pytest. | Crean archivos de verdad y comprueban el resultado en disco. |

**Por dónde te recomiendo empezar a leer:** `core/scanner.py` (el algoritmo central), luego `app.py` y `ui/main_window.py` (la carcasa de la ventana; cada pestaña es un componente en su `pestana_*.py`). El resto son piezas que se entienden solas con su docstring.

## 1.1. Mapa de módulos

| core/ | |
|---|---|
| hashing.py | hash BLAKE2b (parcial y completo); formatear bytes a KB/MB. |
| scanner.py | escanear(): las 3 pasadas. Devuelve ResultadoEscaneo. |
| filtros.py | FiltroEscaneo: qué incluir/excluir. CATEGORIAS_EXTENSIONES. |
| seleccion.py | SeleccionDuplicados: estado conservar/mover por grupo. |
| quarantine.py | mover_a_cuarentena(): mover preservando estructura + CSV. |
| perceptual.py | fotos parecidas: dHash, distancia de Hamming, BK-tree. |
| mover.py | primitivas mover/copiar lotes (las usa reunir.py). |
| reunir.py | listar imágenes + estrategias por archivos / carpeta entera. |
| historial.py | registro de operaciones en JSON + deshacer. |
| config.py | JSON persistente: recuerda el modo (Sencilla/Avanzada) y el tema. |

| ui/ | |
|---|---|
| main_window.py | VentanaPrincipal: cabecera, tema y modo. Crea los 3 componentes. |
| pestana_exactos.py | PestanaExactos: pestaña Duplicados exactos. |
| pestana_parecidas.py | PestanaParecidas: pestaña Fotos parecidas. |
| pestana_reunir.py | PestanaReunir: pestaña Reunir imágenes. |
| tipos_duplicados_dialog.py | VentanaTipos: duplicados por extensión. |
| results_table.py | TablaResultados: la tabla de duplicados exactos. |
| image_viewer.py | VisorImagen: ver una foto a pantalla completa con zoom. |
| history_window.py | VentanaHistorial: lista de operaciones + Deshacer. |
| tipos_dialog.py | diálogo 'qué hacer con los no-imagen'. |

| ui/ | |
|---|---|
| thumbnails.py | generar miniaturas (imagen real o placeholder con la ext.). |
| widgets.py, styles.py, icons.py | piezas visuales, paleta clara/oscura y glifos. |

# 2. El núcleo, explicado

## 2.1. Detectar duplicados exactos: tres pasadas

Comparar el contenido de cada archivo con el de todos los demás sería lentísimo. En vez de eso filtro de lo barato a lo caro (`core/scanner.py`). Cada pasada es una función (`_pasada_tamanos`, `_pasada_hash_parcial` y `_pasada_hash_completo`) y `escanear()` solo las encadena:

- **Pasada 1 — tamaños.** Recorro la carpeta con `os.walk` y agrupo los archivos por su tamaño en bytes. Un tamaño único no puede tener duplicados, así que lo descarto sin abrir el archivo.
- **Pasada 2 — hash parcial.** Para los que comparten tamaño, calculo el hash BLAKE2b de solo los primeros 64 KB. Rápido, y descarta casi todos los que no son iguales.
- **Pasada 3 — hash completo.** Solo para los que además comparten hash parcial, hasheo el archivo entero. Los que coinciden aquí son idénticos byte a byte.

```
# clave = (tamaño, hash_parcial): dos archivos con el mismo hash
# parcial pero distinto tamaño nunca son el mismo archivo.
por_parcial[(tamano, hp)].append(ruta)
...
finalistas = {c: r for c, r in por_parcial.items() if len(r) > 1}
```

> **Nota.** Optimización sutil: si un archivo pesa menos de 64 KB, su hash parcial ya cubre todo el contenido, así que me salto la Pasada 3 para él.

### Podar carpetas con os.walk

`os.walk` entrega tres cosas por carpeta: (ruta, subcarpetas, archivos). Si modificas la lista de subcarpetas *en el sitio*, os.walk no entra en las que quites — así se excluyen `node_modules`, `$RECYCLE.BIN`, etc. sin recorrerlas:

```
subdirs[:] = [d for d in subdirs if not filtro.excluye_carpeta(d)]
```

## 2.2. El hash: BLAKE2b y lectura en bloques

Uso BLAKE2b (viene con Python, más rápido que SHA-256). El archivo completo lo leo de 64 KB en 64 KB para no cargarlo entero en memoria:

```
h = hashlib.blake2b()
with open(ruta, 'rb') as f:
    while True:
        bloque = f.read(65536)     # 64 KB
        if not bloque:              # b'' = fin del archivo
            break
        h.update(bloque)            # acumula
return h.hexdigest()
```

## 2.3. El filtro: lista blanca y lista negra

`FiltroEscaneo` tiene dos mecanismos. Una **lista blanca** (`extensiones_incluidas`): si no está vacía, solo pasan esas extensiones ('buscar solo imágenes'). Y varias **listas negras** (extensiones, carpetas y nombres a ignorar, más un tamaño mínimo). El escáner llama a `filtro.excluye_nombre(...)` y `filtro.excluye_carpeta(...)` y no cuenta ni lee lo que se excluye.

## 2.4. La decisión: qué copia se conserva

`SeleccionDuplicados` es un diccionario `(indice_grupo, ruta) -> 'conservar' | 'mover'`. Empieza todo en 'conservar' (no se movería nada). La regla que impone la app: para poder mover, cada grupo debe tener **exactamente una** copia en 'conservar'.

En modo Sencilla, `preseleccionar_principales()` elige sola la copia principal de cada grupo con esta puntuación (gana la tupla más pequeña):

```
(parece_copia,   # 0 si el nombre no tiene 'copia', '(1)'...  1 si sí
 profundidad,    # nº de carpetas: cuanto menos anidada, mejor
 mtime,          # fecha: la más antigua suele ser el original
 len(ruta))      # a igualdad de todo, la ruta más corta
```

## 2.5. Fotos parecidas: dHash + BK-tree + Union-Find

Aquí no puedo comparar contenido (dos fotos parecidas tienen bytes distintos). Por cada imagen calculo una **huella** (`core/perceptual.py`):

### dHash de 64 bits

Reduzco la imagen a una miniatura de 9×8 en gris. Por cada fila, comparo cada píxel con el de su derecha: bit 1 si sube el brillo, 0 si baja. 8×8 = 64 bits. Mide la *forma*, no el color, por eso aguanta la recompresión y los filtros:

```
bits = 0
for fila in range(8):
    for col in range(8):
        sube = gris[base+col] < gris[base+col+1]
        bits = (bits << 1) | (1 if sube else 0)
```

### Distancia de Hamming

Cómo de distintas son dos huellas = cuántos bits difieren. `bin(a ^ b).count('1')`. 0 = iguales, 64 = opuestas. El deslizador de sensibilidad fija el umbral (~10).

### BK-tree

Para no comparar todas las fotos contra todas, meto los dHash en un BK-tree: un árbol donde cada hijo va etiquetado por su distancia exacta al padre. Al buscar 'los que están a ≤ N', la desigualdad triangular permite **podar** ramas enteras: solo hay que mirar los hijos cuya etiqueta esté en `[d-N, d+N]`.

### Union-Find

Si A se parece a B y B a C, deben acabar en el mismo grupo aunque A y C no se parezcan directamente. Union-Find (conjuntos disjuntos) une esas cadenas: cada elemento apunta a 'otro de su grupo' y siguiendo la cadena se llega al representante. `grupos()` agrupa por representante.

## 2.6. Mover archivos con seguridad

Tres módulos comparten las mismas reglas (`quarantine.py`, `mover.py`, `reunir.py`):

- **Nunca pisar.** Antes de mover, si el destino ya existe pruebo `nombre (1).ext`, `nombre (2).ext`...
- **Preservar estructura.** El destino de cada archivo es `carpeta_destino / os.path.relpath(origen, raiz)`, así se recrea la jerarquía de subcarpetas.
- **Mover, no copiar-y-borrar.** `shutil.move` hace un `rename` atómico si es el mismo disco; solo copia+borra si es otro disco (y ahí el original desaparece cuando la copia ya está a salvo).
- **Destino fuera del origen.** `validar_destino` lo comprueba (si no, el próximo escaneo reencontraría lo movido).

## 2.7. Historial y deshacer

Cada operación de mover/copiar se apunta en `%APPDATA%\BuscadorDuplicados\historial.json` con la lista de pares (origen, destino). `deshacer(id)`:

- si fue **mover**: `shutil.move(destino, origen)` de vuelta — pero si ya hay algo en el origen, lo avisa y no pisa.
- si fue **copiar**: borra las copias del destino. Es el único punto de todo el código donde se borra algo, y solo archivos que la app creó.

# 3. La interfaz, explicada

## 3.1. CustomTkinter y el arranque

CustomTkinter es Tkinter con widgets repintados. `VentanaPrincipal` hereda de `ctk.CTk`. `app.py` es de tres líneas: aplica el tema, crea la ventana y llama a `mainloop()`.

## 3.2. El patrón hilo + cola + after() (el más importante)

Tkinter no es 'thread-safe': solo el hilo principal puede tocar widgets. Pero escanear tarda. La solución, que repito en toda la app:

```
# hilo principal: lanzar y programar el sondeo
threading.Thread(target=self._trabajo, daemon=True).start()
self.after(100, self._revisar_cola)

# hilo secundario: NO toca widgets, solo la cola
def _trabajo(self):
    res = escanear(..., progreso=lambda p: self._cola.put(('prog', p)))
    self._cola.put(('hecho', res))

# hilo principal, cada 100 ms: vaciar la cola y pintar
def _revisar_cola(self):
    try:
        while True:
            tipo, dato = self._cola.get_nowait()
            ...   # actualizar widgets según 'tipo'
    except queue.Empty:
        pass
    self.after(100, self._revisar_cola)
```

Así la ventana no se congela, se ve el progreso y se puede cancelar (el hilo comprueba un `threading.Event` entre archivo y archivo).

## 3.3. Las tres pestañas

Un `ctk.CTkTabview` con tres pestañas. Cada una es un **componente**: una clase que hereda de `ctk.CTkFrame` y guarda su PROPIO estado (las colas, el hilo, el resultado...) en lugar de compartirlo con la ventana. La ventana solo las crea (`self.exactos`, `self.parecidas` y `self.reunir`) y les avisa con `aplicar_modo(avanzado)` y `cancelar_trabajos()`. Cada pestaña repite el patrón: tarjeta de arriba con la carpeta, zona de progreso, panel de estadísticas, galería/tabla y barra inferior con el botón de acción.

## 3.4. Tema claro / oscuro

En `ui/styles.py` cada color es una tupla `(claro, oscuro)`. Los widgets de CustomTkinter aceptan esa tupla y cambian solos. Para lo que NO es CustomTkinter (la tabla `ttk.Treeview` y el `tk.Canvas` del visor) se usa `styles.col(tupla)`, que devuelve el color del tema activo, y se re-pinta a mano sobreescribiendo `_set_appearance_mode`:

```
def _set_appearance_mode(self, mode_string):
    super()._set_appearance_mode(mode_string)
    self._configurar_estilo_ttk()   # ttk no cambia solo
    self._configurar_tags()
```

Los botones deshabilitados pasan a gris (relleno y texto) con `styles.activar_boton(btn, activo)`, para que 'deshabilitado' no se confunda con un color de acción o de error. El tema elegido se guarda en `config.py` (auto/claro/oscuro).

## 3.5. Modo Sencilla / Avanzada

`_aplicar_modo()` no crea ni destruye widgets: los quita y pone del grid con `grid_remove()` / `grid()`. Así el estado (lo que hubiera escrito el usuario) se conserva al alternar. La elección se guarda en `config.py`.

## 3.6. La tabla (ttk.Treeview)

Para los duplicados exactos se usa un `ttk.Treeview` (aguanta miles de filas). Al hacer clic hay que preguntarle a Tk qué hay debajo del ratón:

```
self.tree.identify_row(event.y)      # id de la fila
self.tree.identify_column(event.x)   # '#1' = columna 'Decisión'
self.tree.identify_region(x, y)      # 'heading', 'cell'...
```

Con eso: clic normal = seleccionar (y mostrar la vista previa); clic en 'Decisión' = alternar conservar/mover; doble clic = 'conservar solo esta'.

## 3.7. Miniaturas e imágenes en Tkinter

- Decodificar la imagen (Pillow) se hace en un hilo; produce un objeto PIL.
- Convertir a `ImageTk.PhotoImage` y pegarlo en la fila se hace en el hilo principal.
- **Hay que guardar una referencia** a cada PhotoImage (`self._img_refs[iid] = foto`). Si no, Python la recolecta y la imagen desaparece de la pantalla.

# 4. Empaquetado

`build.spec` configura PyInstaller: un solo archivo, sin consola, con los datos de CustomTkinter incluidos y `icono.ico` como icono. `construir_exe.py` lo ejecuta y comprueba el resultado.

El icono es una lupa **dibujada por código** (`ui/icons.py:icono_ventana`) sobre un `tk.PhotoImage`, y `generar_icono.py` arma el `.ico` multi-tamaño a mano con `struct`. Nada de imágenes externas.

# 5. Cómo se prueba

Mis pruebas no usan *mocks*: cada una crea archivos reales en una carpeta temporal (`tempfile.TemporaryDirectory`), ejecuta la función de verdad y comprueba el estado en disco. Las de interfaz montan la ventana entera, simulan los diálogos (`filedialog` y `messagebox` se sustituyen por funciones que devuelven un valor fijo) y pulsan los métodos de los botones.

```
pytest   # 128 pruebas;  pytest --cov = cobertura (núcleo 93 %, interfaz 79 %)
python pruebas/probar_perceptual.py "D:\Fotos" 10   # con datos reales
```

# 6. Añadir una función nueva: receta

- **1.** Escribe la lógica en un módulo de `core/`, sin tocar la interfaz. Que reciba rutas y devuelva datos.
- **2.** Añade una prueba en `pruebas/` que cree archivos temporales y compruebe el resultado. Súmala a `LOGICA` en `test_suites.py`.
- **3.** En `ui/`, añade los widgets y un handler. Si la tarea es larga, usa el patrón hilo + cola + after().
- **4.** Si mueve o copia archivos, hazlo pasar por `core/mover.py` y regístralo en `historial.py` para que se pueda deshacer.
- **5.** Ejecuta `pytest`, `ruff check .` y `python -m mypy`. Todo en verde antes de dar por hecho nada.